

数据挖掘与机器学习

聚类算法核心概念与应用解析

第四次作业 信管2302 任宇轩

CONTENTS

01

簇的核心概念

明确聚类分析的目标，解析“簇”的基础数学定义与核心内涵。

02

相似度度量

深入学习欧氏距离与余弦相似度，掌握量化数据点相似性的关键方法。

03

K-means 算法

拆解算法核心原理，解析从初始质心选择到迭代优化收敛的完整步骤。

04

层次聚类 (HC)

探究凝聚式与分裂式两种核心策略，理解层次聚类的树状结构生成逻辑与适用场景。

05

算法对比与总结

对比不同算法的优劣与适用场景，通过实战案例与思考练习巩固知识体系。

MODULE 01

簇 (Cluster) 的核心概念



内部高紧密性

同一簇内的样本具有高度相似的特征，数据点间距离尽可能小。



外部强分离性

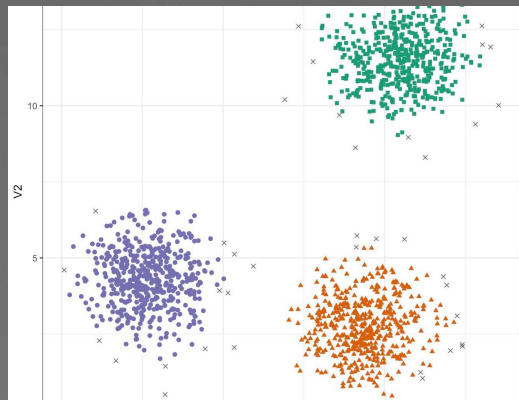
不同簇之间的样本特征差异显著，各类簇在特征空间中相互独立。



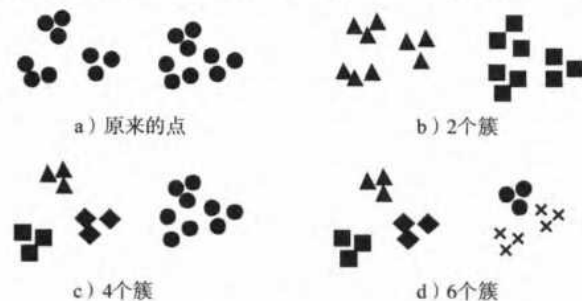
天然无监督性

无需预设标签，算法自动从数据中发现内在的类别结构与规律。

什么是簇？



在许多应用中，簇的概念都没有严格的定义。为了理解确定簇构造的困难性，可参考图 9-1。该图显示了 18 个点和将它们划分成簇的 3 种不同方法。标记的形状指示簇的隶属关系。图 9-1b 和图 9-1d 分别将数据划分成两部分和六部分。然而，将 2 个较大的簇都划分成 3 个子簇可能是人的视觉系统造成的假象。此外，说这些点形成 4 个簇（如图 9-1c 所示）可能也不无道理。该图表明簇的定义是不精确的，而最好的定义依赖于数据的特性和期望的结果。另外，簇的形象表现在空间分布上也不是确定的，而是成各种不同的形状，在二维平面里就可以有各种不同的形状，如图 9-2 所示，在多维空间里，更是有更多的形状。所以簇的定义，也需要具体情况具体分析，但总的趋势是，同一个簇的样本在空间上是靠拢在一起的。



图示为三维数据点的聚类分布，不同颜色代表独立的簇，直观体现了“物以类聚”的数据分布规律。

核心定义：簇是数据对象的集合，这些对象在簇内呈现高度的相似性，而在不同簇之间则表现出显著的相异性，是聚类分析的基本单元。



最大化簇内相似性

通过算法让同一个簇内的成员尽可能“像”，确保组内数据特征高度一致，形成紧密的整体。



最小化簇间相似性

使不同簇的成员尽可能“不像”，建立清晰的簇间边界，确保每个簇都具有独特的数据特征。

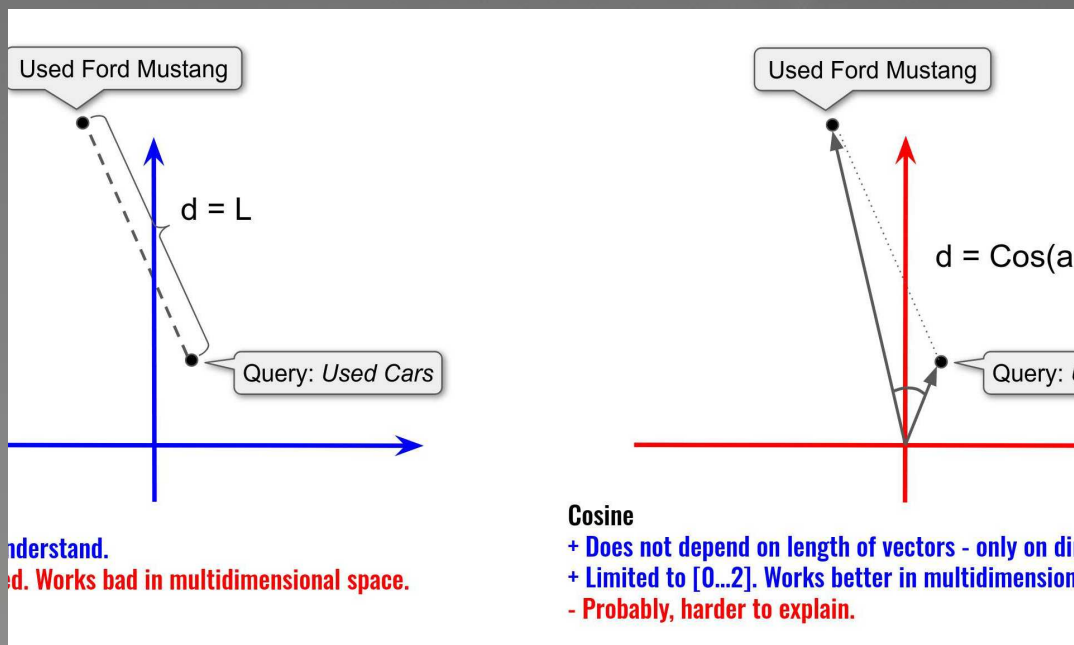
理想的簇应具备：**高内聚性 (High Cohesion)**与**低耦合性 (Low Coupling)**

MODULE 02

相似度的度量方法

从数学视角定义“相似”的核心逻辑，探索距离度量与相似度计算的经典范式，为聚类分析建立科学基石。

常用的两种度量方法



图示直观对比了欧氏距离（关注空间绝对距离）与余弦距离（关注向量夹角）在多维空间中的几何意义，展示了两者核心逻辑的本质区别。

01. 欧氏距离 (Euclidean Distance)

衡量两点间的直线距离，直观易懂，但对数值尺度敏感，需先做标准化处理。适用于连续数值变量的相似度计算，如身高、收入、物理坐标等场景。

02. 余弦距离 (Cosine Distance)

衡量向量夹角的余弦值，核心关注方向而非长度，不受向量尺度影响。广泛应用于非结构化数据场景，如文本相似度分析、推荐系统的用户兴趣匹配等。

03

MODULE

K-means算法详解

K-means算法核心步骤

算法步骤:

- 1) 从 n 个数据对象中任意选择 k 个对象作为初始聚类中心;
- 2) 循环 3) 到 4) 直到每个聚类不再发生变化为止;
- 3) 根据每个聚类对象的均值 (中心对象), 计算每个对象与这些中心对象的距离; 并根据最小距离重新对相应对象进行划分;
- 4) 重新计算每个聚类的均值 (中心对象), 直到聚类中心不再变化。这种划分使得下式最小:

$$E = \sum_{j=1}^k \sum_{x_i \in \omega_j} \|x_i - m_j\|^2$$



01. 初始化

从数据集中随机选择 K 个数据点, 将它们作为初始的簇中心。这是算法的起点, 决定了聚类的初始状态。



02. 分配

计算所有数据点到 K 个簇中心的距离, 将每个点分配给距离最近的中心, 形成 K 个初始的簇。



03. 更新

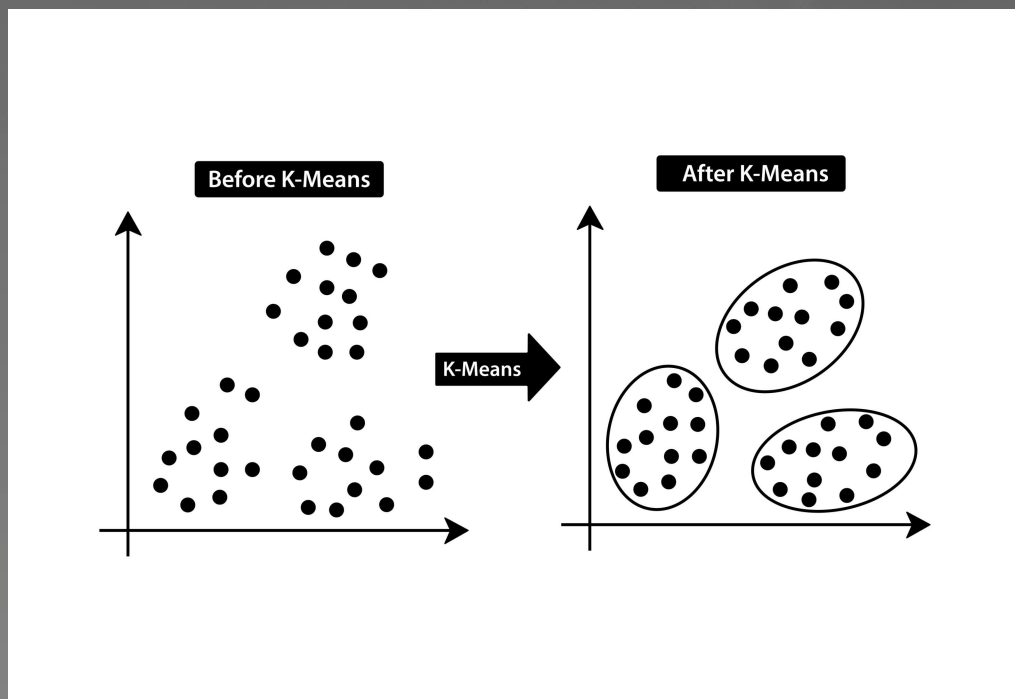
重新计算每个簇中所有数据点的均值 (重心), 并将该均值作为新的簇中心, 实现中心位置的迭代优化。



04. 迭代

不断重复“分配-更新”的过程, 直到簇中心的位置不再发生显著变化, 或者达到预设的最大迭代次数, 算法收敛。

K-means迭代过程可视化



图示直观展现了数据点从散乱分布，经过K-means算法迭代，最终收敛为界限清晰的三类簇群的全过程，体现了算法的动态聚类能力。

01 猜测中心

在数据空间中随机初始化放置K个点作为初始聚类中心，这是算法迭代的起点。

02 划分归属

计算每个数据点到K个中心的距离，将其分配给距离最近的中心，形成K个初步簇群。

03 调整中心

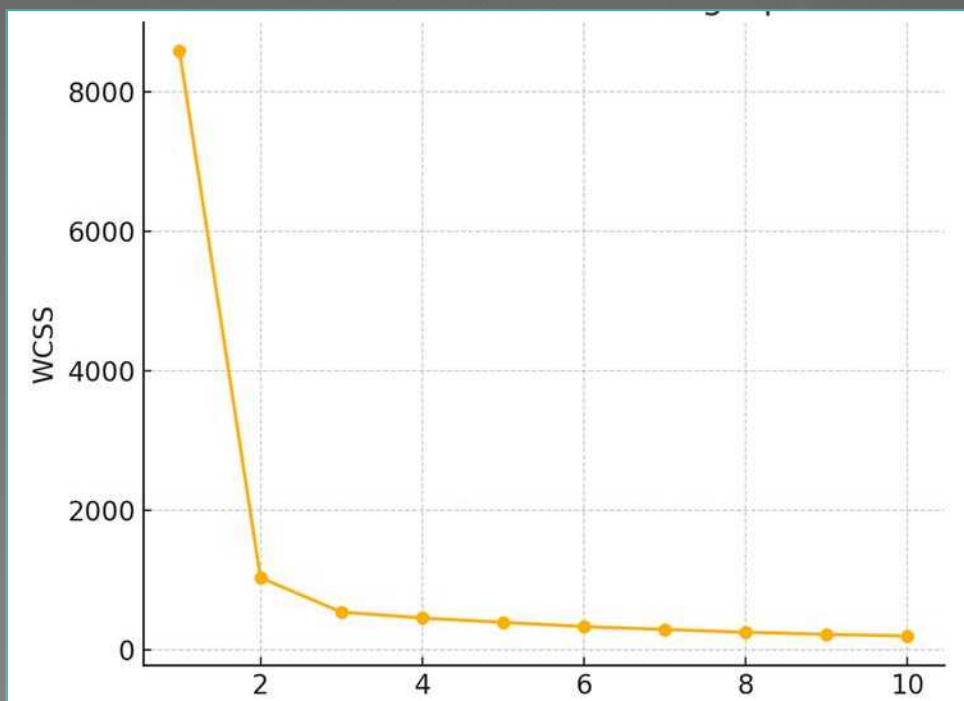
重新计算每个簇内所有数据点的平均值（重心），将中心点移动到该重心位置，优化聚类中心。

04 重复优化

不断重复“划分归属”与“调整中心”的步骤，直到中心点位置不再变化或变化极小，算法收敛。

读书笔记：K-means的局限性

以葡萄酒为例：K-means的局限性



图示为典型的肘部法则曲线，WCSS（簇内误差平方和）随K值增加而递减，在K=2处出现明显的“手肘”拐点。

尽管最近邻算法简单且实用，但是在如下情形中使用该算法可能无法取得好的效果。

- **类别不平衡**：如果待预测的类别有多个，并且在大小方面存在很大的不同，那么那些属于最小类别的数据点可能会被来自更大类别的数据点所掩盖，它们被错误分类的风险更大。为了提高准确度，可以使用加权投票法来取代少数服从多数的原则，这会确保较近数据点类别的权重比较远的更大。
- **预测变量过多**：如果待考虑的预测变量太多，在多个维度上识别和处理近邻会导致计算量大增。而且，有些预测变量可能是多余的，它们对提高预测准确度没有用处。为了解决这个问题，可以使用第3章介绍的降维技巧，只抽取最具影响力的预测变量用于分析。



MODULE 04

层次聚类(HC)算法详解

不同于K-means的扁平划分，层次聚类通过构建嵌套的聚类树（树状图）揭示数据的层级结构，为探索数据内在的嵌套关系提供了更直观、更深入的视角。

HC聚类的两种思想



凝聚式 (Agglomerative) · 自底向上

每个数据点最初自成一类，算法不断寻找并合并当前最相似的两个簇，这一过程持续迭代，直到所有数据点最终聚合成一个统一的大簇。如同个体逐步联合形成组织的过程。

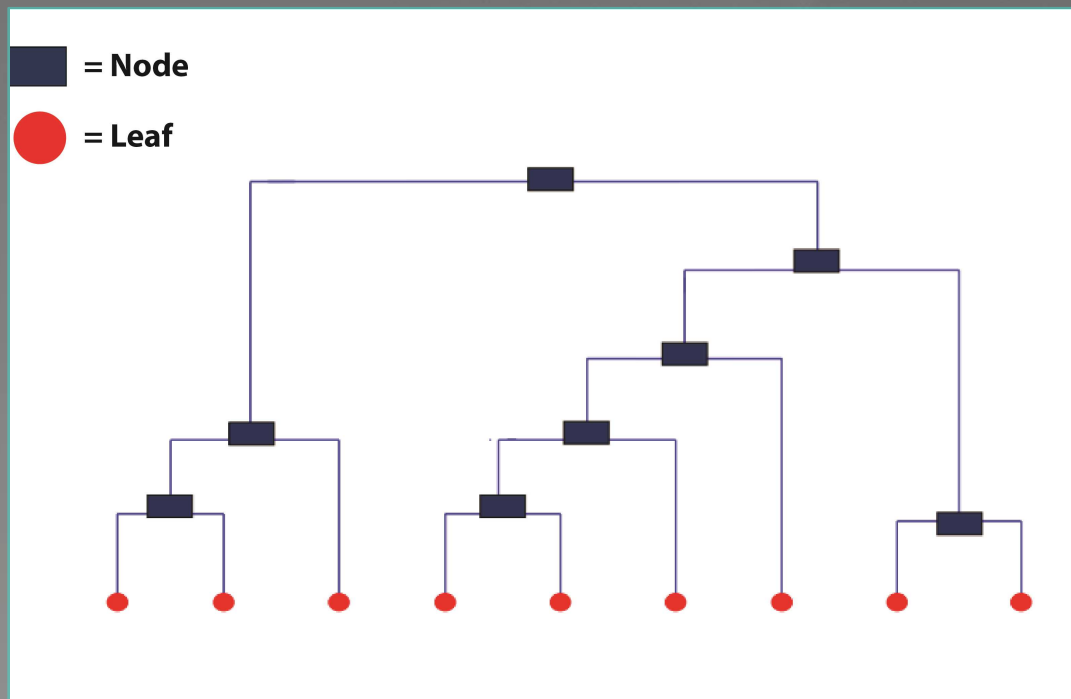


分裂式 (Divisive) · 自顶向下

所有数据点一开始被视为一个整体大簇，算法持续从中分裂出最大或最不相似的子簇，直至每个数据点都成为独立的一类。这与凝聚式的路径完全相反，是从整体到个体的拆解。

在实际应用中，**凝聚式层次聚类**因实现更简单、计算效率更优，是目前使用更为广泛的层次聚类方法。

HC聚类结果图解读：树状图 (Dendrogram)



树状图通过层级结构直观展示数据点的合并过程，每一个节点的高度代表了簇间的相似性或距离，是层次聚类最经典的可视化呈现方式。



横轴：数据点分布

代表数据集中的各个样本点或初始数据对象，是聚类分析的基础单元，展示了数据的原始构成与分布情况。



纵轴：簇间距离度量

表示不同簇之间的相似性或距离大小。节点在纵轴上的位置越高，意味着合并的两个簇原本的差异越大，相似度越低。



如何确定分类数量？

在纵轴上绘制一条水平直线，该线与图中垂直线相交的次数，即为最终确定的聚类数量，操作直观且易于理解。

算法对比与总结

K-means 算法



划分式聚类：一次性生成K个互不相交的簇，将数据点直接分配至最近的簇中心，结构清晰直接。



需预定义K值，高效且稳定：算法复杂度较低($O(nkt)$)，非常适合大规模数据集，但聚类结果会受初始中心选择的影响。

层次聚类 (Hierarchical Clustering)



层次嵌套结构：通过凝聚或分裂的方式，生成树状的聚类结构（谱系图），能揭示数据间的层级关系。



无需预定义K值，结果稳健：无需提前指定聚类数量，结果稳定不受初始值影响，但时间复杂度高($O(n^3)$)，适合中小规模数据。

核心总结：K-means 追求高效与规模，适合探索性大数据分析；层次聚类追求结构与解释性，适合深入挖掘数据内在关联。

Q & A

谢谢！